

## Laporan Final Project

### *SISTEM BASIS DATA BIOSKOP CINETRACK:*

Perancangan, Implementasi, dan Analisis Normalisasi Basis Data  
Operasional Bioskop Multi-Cabang



#### Anggota Kelompok:

|                       |            |
|-----------------------|------------|
| Maleka Ghaniya        | 5025241189 |
| Jorell Ramos Sinaga   | 5025241202 |
| Angela Vania Sugiyono | 5025241226 |

Departemen Teknik Informatika, FTEIC  
Institut Teknologi Sepuluh Nopember

2026

# Daftar Isi

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Studi Kasus</b>                                                              | <b>5</b>  |
| 1.1      | Latar Belakang . . . . .                                                        | 5         |
| 1.2      | Studio dan Film . . . . .                                                       | 5         |
| 1.3      | Jadwal Tayang . . . . .                                                         | 5         |
| 1.4      | Transaksi dan Pembayaran . . . . .                                              | 5         |
| 1.5      | Kebutuhan Laporan . . . . .                                                     | 6         |
| <b>2</b> | <b>Perancangan Basis Data</b>                                                   | <b>7</b>  |
| 2.1      | Entity Relationship Diagram (ERD) . . . . .                                     | 7         |
| 2.2      | Conceptual Data Model (CDM) . . . . .                                           | 7         |
| 2.3      | Physical Data Model (PDM) . . . . .                                             | 8         |
| <b>3</b> | <b>Analisis Normalisasi hingga Bentuk Normal Ketiga (3NF)</b>                   | <b>10</b> |
| 3.1      | Definisi 3NF . . . . .                                                          | 10        |
| 3.2      | Analisis Per Tabel . . . . .                                                    | 10        |
| 3.3      | Keputusan Desain yang Mendukung 3NF . . . . .                                   | 12        |
| 3.3.1    | Pemisahan Entitas <i>genre</i> dari <i>film</i> . . . . .                       | 12        |
| 3.3.2    | Pemisahan Entitas <i>kategori</i> dari <i>produk_kantin</i> . . . . .           | 12        |
| 3.3.3    | Tabel <i>Junction</i> <i>produk_kantin_transaksi</i> . . . . .                  | 13        |
| 3.3.4    | Atribut <i>Snapshot</i> : <i>harga_beli</i> dan <i>subtotal</i> . . . . .       | 13        |
| 3.3.5    | Pemisahan Relasi <i>pegawai-shift</i> menjadi <i>Many-to-Many</i> . . . . .     | 13        |
| 3.4      | Kesimpulan Normalisasi . . . . .                                                | 13        |
| <b>4</b> | <b>Implementasi SQL dan Data Record</b>                                         | <b>14</b> |
| <b>5</b> | <b>Implementasi SQL Query</b>                                                   | <b>15</b> |
| 5.1      | Kasus Query pada Sistem CineTrack . . . . .                                     | 15        |
| 5.2      | Kasus 1: Menampilkan Film <i>Now Showing</i> Beserta Genre-nya . . . . .        | 18        |
| 5.3      | Kasus 2: Menampilkan Jadwal Tayang Lengkap . . . . .                            | 19        |
| 5.4      | Kasus 3: Menampilkan Transaksi dengan Total Tagihan di Atas Rp200.000 . . . . . | 20        |
| 5.5      | Kasus 4: Rekap Penjualan Produk Kantin per Kategori . . . . .                   | 21        |
| 5.6      | Kasus 5: Menampilkan Pelanggan yang Belum Pernah Bertransaksi . . . . .         | 22        |
| <b>6</b> | <b>Implementasi SQL <i>Function</i></b>                                         | <b>23</b> |

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| 6.1       | <i>Function 1: GetTotalTiketByPelanggan</i> . . . . .     | 23        |
| 6.2       | <i>Function 2: GetPendapatanByStudio</i> . . . . .        | 24        |
| 6.3       | <i>Function 3: GetDurasiTayang</i> . . . . .              | 25        |
| 6.4       | <i>Function 4: CekStatusKursi</i> . . . . .               | 27        |
| 6.5       | <i>Function 5: CekDiskonMember</i> . . . . .              | 27        |
| <b>7</b>  | <b>Implementasi <i>Stored Procedure</i></b>               | <b>29</b> |
| 7.1       | Kasus 1: <i>NaikkanHargaInflasi</i> . . . . .             | 29        |
| 7.2       | Kasus 2: <i>TambahJadwalMidnight</i> . . . . .            | 30        |
| 7.3       | Kasus 3: <i>TambahStokKantin</i> . . . . .                | 31        |
| 7.4       | Kasus 4: <i>BatalTransaksi</i> . . . . .                  | 32        |
| 7.5       | Kasus 5: <i>UpdateStatusFilmHarian</i> . . . . .          | 33        |
| <b>8</b>  | <b>Implementasi <i>Trigger</i></b>                        | <b>34</b> |
| 8.1       | Kasus 1: Pencegahan <i>Double-Booking</i> Kursi . . . . . | 34        |
| 8.2       | Kasus 2: Auto-Kalkulasi Harga Beli Tiket . . . . .        | 35        |
| 8.3       | Kasus 3: Pengurangan Stok Kantin Otomatis . . . . .       | 35        |
| 8.4       | Kasus 4: Pencegahan Stok Kantin Minus . . . . .           | 36        |
| 8.5       | Kasus 5: Pencegahan Jadwal Tayang Bentrok . . . . .       | 37        |
| <b>9</b>  | <b>Implementasi <i>Indexing</i></b>                       | <b>38</b> |
| 9.1       | Deskripsi . . . . .                                       | 38        |
| 9.2       | <i>Index 1: idx_film_status_tayang</i> . . . . .          | 38        |
| 9.3       | <i>Index 2: idx_jtft_film</i> . . . . .                   | 39        |
| 9.4       | <i>Index 3: idx_transaksi_total_tagihan</i> . . . . .     | 39        |
| 9.5       | <i>Index 4: idx_tiket_jadwal_kursi</i> . . . . .          | 40        |
| 9.6       | <i>Index 5: idx_pelanggan_email</i> . . . . .             | 40        |
| <b>10</b> | <b>Demo Aplikasi Web CineTrack</b>                        | <b>42</b> |
| <b>11</b> | <b>Kesimpulan</b>                                         | <b>42</b> |

## Daftar Tabel

|   |                                                         |    |
|---|---------------------------------------------------------|----|
| 1 | Analisis Normalisasi 3NF Basis Data CineTrack . . . . . | 10 |
| 2 | Kasus Query pada Sistem Basis Data CineTrack . . . . .  | 16 |
| 3 | Daftar <i>Index</i> pada Basis Data CineTrack . . . . . | 38 |

## Daftar Gambar

|    |                                                                                    |    |
|----|------------------------------------------------------------------------------------|----|
| 1  | ERD Sistem Basis Data Bioskop CineTrack . . . . .                                  | 7  |
| 2  | CDM Sistem Basis Data Bioskop CineTrack . . . . .                                  | 8  |
| 3  | PDM Sistem Basis Data Bioskop CineTrack . . . . .                                  | 9  |
| 4  | Hasil Query Kasus 1 — Film <i>Now Showing</i> Beserta Genre . . . . .              | 19 |
| 5  | Hasil Query Kasus 2 — Jadwal Tayang Lengkap . . . . .                              | 20 |
| 6  | Hasil Query Kasus 3 — Transaksi di Atas Rp200.000 . . . . .                        | 21 |
| 7  | Hasil Query Kasus 4 — Rekap Penjualan Produk Kantin per Kategori .                 | 21 |
| 8  | Hasil Query Kasus 5 — Pelanggan Belum Pernah Bertransaksi . . . . .                | 22 |
| 9  | Hasil Eksekusi <i>Function 1</i> — <code>GetTotalTiketByPelanggan</code> . . . . . | 24 |
| 10 | Hasil Eksekusi <i>Function 2</i> — <code>GetPendapatanByStudio</code> . . . . .    | 25 |
| 11 | Hasil Eksekusi <i>Function 3</i> — <code>GetDurasiTayang</code> . . . . .          | 26 |
| 12 | Harga Tiket Sebelum Eksekusi <code>NaikkanHargaInflasi</code> . . . . .            | 29 |
| 13 | Harga Makanan Sebelum Eksekusi <code>NaikkanHargaInflasi</code> . . . . .          | 29 |
| 14 | Harga Tiket Sesudah Eksekusi <code>NaikkanHargaInflasi</code> . . . . .            | 30 |
| 15 | Harga Makanan Sesudah Eksekusi <code>NaikkanHargaInflasi</code> . . . . .          | 30 |
| 16 | Hasil Eksekusi <code>TambahJadwalMidnight</code> . . . . .                         | 31 |
| 17 | Stok Kantin Sebelum Eksekusi <code>TambahStokKantin</code> . . . . .               | 32 |
| 18 | Stok Kantin Sesudah Eksekusi <code>TambahStokKantin</code> . . . . .               | 32 |
| 19 | Uji Coba <i>Insert</i> Tiket Baru (Kondisi Berhasil) . . . . .                     | 34 |
| 20 | Pesan <i>Error</i> Saat Terjadi Indikasi <i>Double Booking</i> . . . . .           | 35 |
| 21 | Hasil Kalkulasi Otomatis Harga Tiket pada Basis Data . . . . .                     | 35 |
| 22 | Stok Produk Kantin Sebelum Terjadi Transaksi . . . . .                             | 36 |
| 23 | Stok Produk Kantin Berkurang Secara Otomatis Setelah Transaksi . .                 | 36 |
| 24 | Hasil EXPLAIN <i>Index 1</i> . . . . .                                             | 39 |
| 25 | Hasil EXPLAIN <i>Index 2</i> . . . . .                                             | 39 |
| 26 | Hasil EXPLAIN <i>Index 3</i> . . . . .                                             | 40 |
| 27 | Hasil EXPLAIN <i>Index 4</i> . . . . .                                             | 40 |
| 28 | Hasil EXPLAIN <i>Index 5</i> . . . . .                                             | 41 |

## **Daftar Isi**

## **Daftar Tabel**

## **Daftar Gambar**

# 1 Studi Kasus

## 1.1 Latar Belakang

CineTrack adalah bioskop lokal yang telah beroperasi lebih dari satu dekade, namun hingga kini seluruh proses operasionalnya, penjualan tiket, pencatatan jadwal tayang, hingga pemantauan stok kantin masih dikelola secara manual. Seiring bertambahnya jumlah pengunjung dan studio, cara ini mulai menimbulkan masalah serius seperti kursi terjual ganda karena tidak ada sinkronisasi antar kasir, laporan pendapatan harian sulit disusun karena data tersebar di berbagai catatan terpisah, dan manajemen tidak memiliki gambaran yang jelas tentang status penayangan film untuk keperluan promosi. Oleh karena itu, manajemen CineTrack memutuskan untuk membangun sistem basis data terpusat yang mengintegrasikan seluruh kebutuhan operasional bioskop. Lebih dari itu, CineTrack yang awalnya beroperasi sebagai bioskop tunggal kini tengah merencanakan ekspansi dengan membuka cabang di beberapa kota besar di Indonesia. Untuk mendukung pertumbuhan ini, sistem basis data yang dirancang perlu mengakomodasi pengelolaan multi-cabang, di mana setiap cabang memiliki studio, jadwal tayang, dan operasional tersendiri namun tetap terintegrasi dalam satu sistem terpusat milik manajemen pusat CineTrack.

## 1.2 Studio dan Film

Bioskop ini memiliki beberapa studio, masing-masing dengan kapasitas berbeda dan kursi yang dibagi ke dalam tipe reguler, VIP, dan *couple seat*, tiap tipe memiliki tarif sendiri dan posisi kursi dicatat berdasarkan baris dan kolom. Setiap film yang ditayangkan menyimpan informasi seperti judul, genre, durasi, rating usia, sinopsis, sutradara, dan pemeran utama, serta memiliki status penayangan yang dapat berubah antara *coming soon*, *now showing*, dan *leaving soon*, status ini digunakan oleh tim promosi sebagai dasar pembuatan poster dan materi iklan.

## 1.3 Jadwal Tayang

Setiap hari disusun jadwal tayang yang menentukan film mana ditayangkan di studio mana, pada tanggal dan jam berapa; satu studio dapat memiliki beberapa sesi dalam sehari dengan film yang berbeda. Harga dasar tiket per sesi dapat bervariasi tergantung waktu tayang dan hari. Sesi malam dan akhir pekan umumnya dikenakan tarif lebih tinggi.

## 1.4 Transaksi dan Pembayaran

Saat pelanggan datang ke loket, kasir menampilkan peta kursi yang tersedia untuk sesi yang dipilih; pelanggan dapat memesan satu atau lebih kursi sekaligus dan menambahkan pembelian makanan atau minuman kantin dalam transaksi yang sama, di mana setiap produk kantin memiliki nama, kategori, harga satuan, dan stok tercatat. Total tagihan merupakan akumulasi harga tiket dan produk kantin; pembayaran dapat dilakukan tunai maupun non-tunai. Data pelanggan seperti nama, nomor telepon, dan surel ikut dicatat, begitu pula identitas kasir yang menangani transaksi beserta *shift* kerjanya.

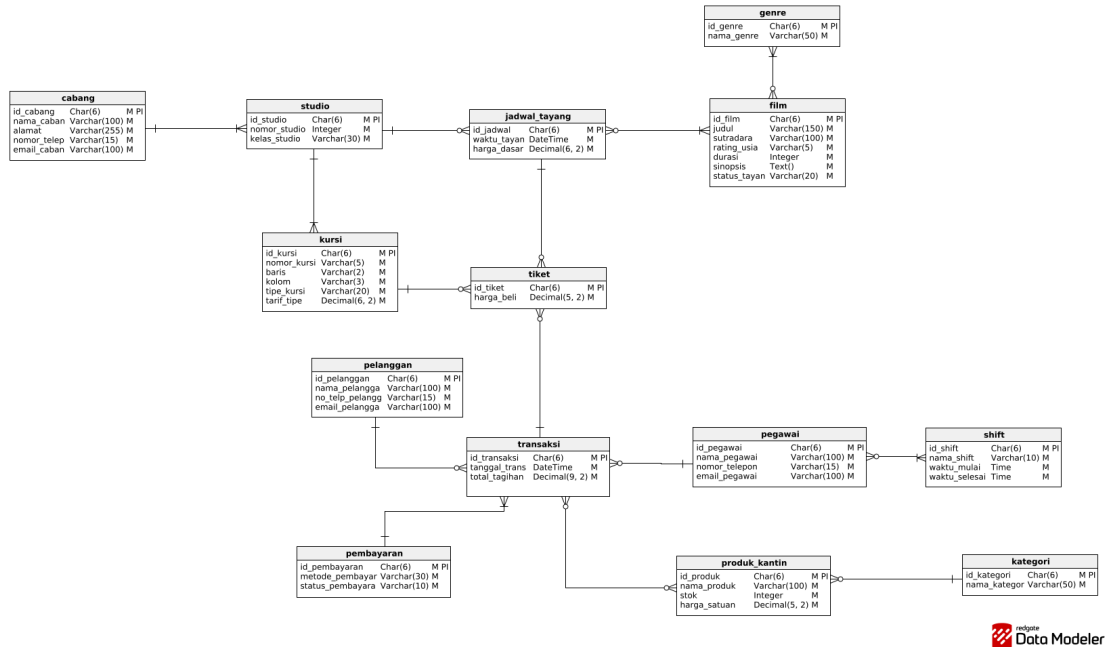
## 1.5 Kebutuhan Laporan

Pada akhir hari, manajer membutuhkan laporan yang merangkum total pendapatan tiket per studio dan per film, jumlah kursi terisi per sesi tayang, serta produk kantin paling laris; riwayat transaksi per pelanggan juga perlu tersimpan sebagai dasar program loyalitas ke depannya. Berdasarkan seluruh kebutuhan ini, manajemen CineTrack menugaskan tim untuk merancang dan mengimplementasikan sistem basis data yang mencakup perancangan struktur data, *query* pelaporan, fungsi kalkulasi otomatis, serta prosedur transaksi yang aman dan konsisten.





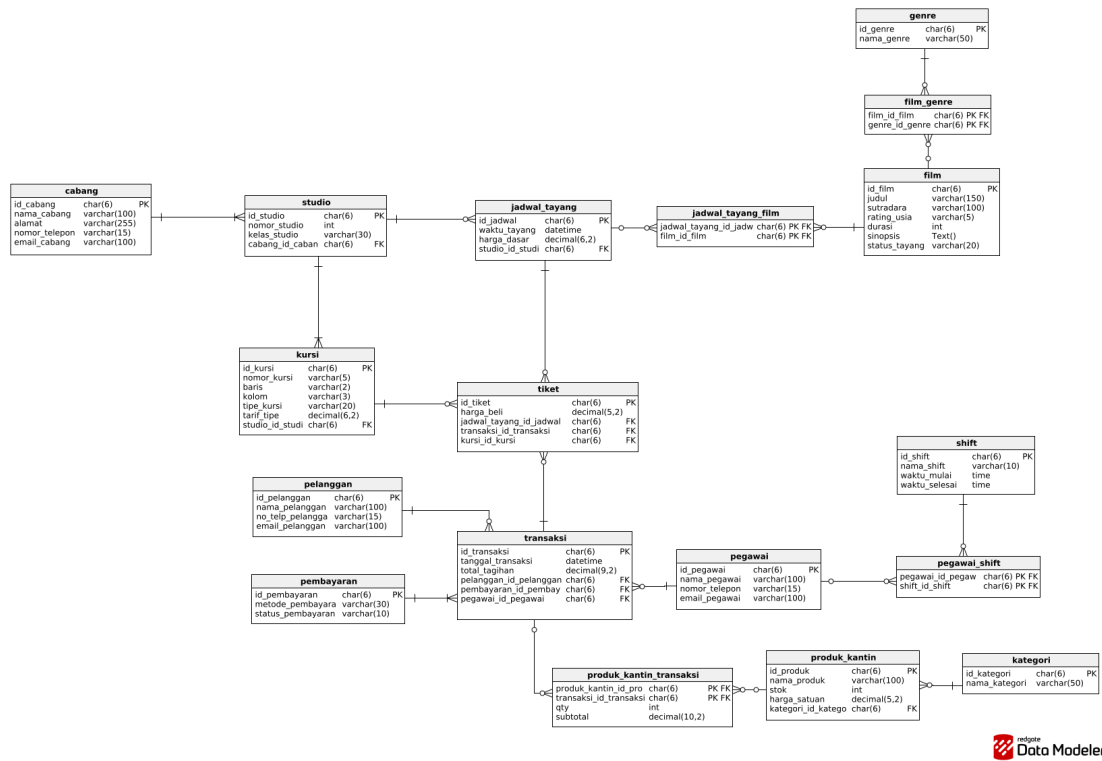
kardinalitas relasinya. Relasi *many-to-many* seperti film–genre, jadwal–film, pegawai–shift, dan transaksi–produk kantin masih direpresentasikan sebagai relasi langsung antar entitas tanpa tabel *junction*, sesuai dengan sifat model konseptual yang belum menyentuh detail implementasi fisik.



Gambar 2: CDM Sistem Basis Data Bioskop CineTrack

## 2.3 Physical Data Model (PDM)

PDM merupakan hasil transformasi CDM ke dalam bentuk fisik yang siap diimplementasikan pada sistem manajemen basis data. Pada tahap ini, seluruh relasi *many-to-many* ditransformasikan menjadi tabel *junction* eksplisit, yaitu *film\_genre*, *jadwal\_tayang\_film*, *pegawai\_shift*, dan *produk\_kantin\_transaksi*. Seluruh *foreign key* dan tipe data juga telah didefinisikan secara eksplisit, sehingga total terdapat 18 tabel pada PDM.



Data Modeler

Gambar 3: PDM Sistem Basis Data Bioskop CineTrack

### 3 Analisis Normalisasi hingga Bentuk Normal Ketiga (3NF)

#### 3.1 Definisi 3NF

Suatu relasi dikatakan berada dalam Bentuk Normal Ketiga (3NF) apabila memenuhi tiga syarat berikut:

1. Berada dalam 1NF: setiap atribut bernilai atomik dan tidak ada grup atribut yang berulang.
2. Berada dalam 2NF: setiap atribut non-*key* bergantung secara penuh pada *primary key* (tidak ada *partial dependency*).
3. Tidak terdapat *transitive dependency*: atribut non-*key* tidak bergantung pada atribut non-*key* lainnya.

#### 3.2 Analisis Per Tabel

Berikut adalah analisis pemenuhan 3NF untuk setiap tabel pada basis data CineTrack.

Tabel 1: Analisis Normalisasi 3NF Basis Data CineTrack

| No | Tabel  | Primary Key | Atribut Non-Key                                                                                                                                                                                                                            | 3NF |
|----|--------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 1  | cabang | id_cabang   | nama_cabang, alamat, nomor_telepon, email_cabang — semua bergantung penuh pada id_cabang, tidak ada dependensi transitif.                                                                                                                  | ✓   |
| 2  | studio | id_studio   | nomor_studio, kelas_studio — bergantung penuh pada id_studio. FK cabang_id_cabang merupakan referensi eksternal, bukan dependensi transitif.                                                                                               | ✓   |
| 3  | kursi  | id_kursi    | nomor_kursi, baris, kolom, tipe_kursi, tarif_tipe — semua bergantung penuh pada id_kursi. tarif_tipe adalah tarif per tipe kursi yang merupakan properti intrinsik kursi, bukan atribut yang bergantung pada atribut non- <i>key</i> lain. | ✓   |

(bersambung ke halaman berikutnya)

| No | Tabel              | Primary Key                                      | Atribut Non-Key                                                                                                                                                       | 3NF |
|----|--------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 4  | jadwal_tayang      | id_jadwal                                        | waktu_tayang, harga_dasar — keduanya merupakan properti sesi tayang yang bergantung penuh pada id_jadwal. Tidak ada dependensi transitif.                             | ✓   |
| 5  | film               | id_film                                          | judul, durasi, sinopsis, rating_usia, sutradara, status_tayang — semua bergantung penuh pada id_film. Genre dipisah ke tabel film_genre untuk menghindari redundansi. | ✓   |
| 6  | genre              | id_genre                                         | nama_genre — bergantung penuh pada id_genre.                                                                                                                          | ✓   |
| 7  | film_genre         | film_id_film, genre_id_genre (komposit)          | Tidak ada atribut non-key. Tabel <i>junction</i> murni untuk relasi <i>many-to-many</i> film-genre.                                                                   | ✓   |
| 8  | jadwal_tayang_film | jadwal_tayang_id_jadwal, film_id_film (komposit) | Tidak ada atribut non-key. Tabel <i>junction</i> untuk relasi <i>many-to-many</i> jadwal-film.                                                                        | ✓   |
| 9  | tiket              | id_tiket                                         | harga_beli — merupakan <i>snapshot</i> harga saat transaksi terjadi, bergantung pada id_tiket. FK id_jadwal, id_transaksi, id_kursi adalah referensi eksternal.       | ✓   |
| 10 | pelanggan          | id_pelanggan                                     | nama_pelanggan, no_telp_pelanggan, email_pelanggan — semua bergantung penuh pada id_pelanggan.                                                                        | ✓   |
| 11 | pegawai            | id_pegawai                                       | nama_pegawai, nomor_telepon, email_pegawai — semua bergantung penuh pada id_pegawai.                                                                                  | ✓   |
| 12 | shift              | id_shift                                         | nama_shift, waktu_mulai, waktu_selesai — bergantung penuh pada id_shift.                                                                                              | ✓   |
| 13 | pegawai_shift      | pegawai_id_pegawai, shift_id_shift (komposit)    | Tidak ada atribut non-key. Tabel <i>junction</i> untuk relasi <i>many-to-many</i> pegawai-shift.                                                                      | ✓   |

(bersambung ke halaman berikutnya)

| No | Tabel                   | Primary Key                                                | Atribut Non-Key                                                                                                                                                                                            | 3NF |
|----|-------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 14 | pembayaran              | id_pembayaran                                              | metode_pembayaran, status_pembayaran — bergantung penuh pada id_pembayaran.                                                                                                                                | ✓   |
| 15 | transaksi               | id_transaksi                                               | tanggal_transaksi, total_tagihan — bergantung penuh pada id_transaksi. total_tagihan disimpan sebagai <i>snapshot</i> akumulasi untuk keperluan audit, bukan <i>derived attribute</i> yang dihitung ulang. | ✓   |
| 16 | produk_kantin           | id_produk                                                  | nama_produk, stok, harga_satuan — bergantung penuh pada id_produk. Kategori dipisah ke entitas kategori untuk menghindari redundansi nilai berulang.                                                       | ✓   |
| 17 | kategori                | id_kategori                                                | nama_kategori — bergantung penuh pada id_kategori.                                                                                                                                                         | ✓   |
| 18 | produk_kantin_transaksi | produk_kantin_id_produk, transaksi_id_transaksi (komposit) | qty (INT) dan subtotal (DECIMAL(10,2)) — keduanya bergantung penuh pada kombinasi PK komposit. subtotal merupakan <i>snapshot</i> hasil $qty \times harga\_satuan$ saat transaksi.                         | ✓   |

### 3.3 Keputusan Desain yang Mendukung 3NF

Beberapa keputusan desain penting yang dilakukan untuk memastikan pemenuhan 3NF adalah sebagai berikut:

#### 3.3.1 Pemisahan Entitas genre dari film

Apabila *genre* disimpan sebagai atribut Varchar langsung di *film*, nilai seperti “Action” akan berulang di banyak baris — melanggar prinsip minimasi redundansi. Lebih lanjut, satu film dapat memiliki lebih dari satu genre (*many-to-many*), sehingga diperlukan tabel *junction* *film\_genre* dengan PK komposit (*film\_id\_film*, *genre\_id\_genre*).

#### 3.3.2 Pemisahan Entitas kategori dari produk\_kantin

Serupa dengan genre, nilai kategori produk kantin (seperti “Makanan” dan “Minuman”) akan berulang di setiap baris tabel *produk\_kantin* apabila disimpan sebagai Varchar. Dengan memisahkannya ke entitas *kategori*, redundansi dieliminasi dan konsistensi nilai terjaga.

### 3.3.3 Tabel *Junction* produk\_kantin\_transaksi

Satu transaksi dapat mencakup pembelian banyak produk kantin dengan kuantitas berbeda-beda. Apabila `id_produk` disimpan langsung sebagai FK di `transaksi`, hanya satu produk yang dapat dicatat per transaksi. Tabel *junction* ini memungkinkan pencatatan `qty` dan `subtotal` per produk per transaksi, sekaligus memenuhi 2NF karena kedua atribut tersebut bergantung penuh pada kombinasi PK komposit.

### 3.3.4 Atribut *Snapshot*: harga\_beli dan subtotal

Nilai `harga_beli` pada `tiket` dan `subtotal` pada `produk_kantin_transaksi` secara konseptual bersifat *derived*, namun sengaja disimpan sebagai atribut fisik. Hal ini dilakukan untuk menjaga integritas data historis — apabila `tarif_tipe` atau `harga_satuan` berubah di kemudian hari, harga yang telah dibayarkan tetap tercatat dengan benar. Pendekatan ini tidak melanggar 3NF karena nilai-nilai tersebut bergantung penuh pada PK tabelnya masing-masing.

### 3.3.5 Pemisahan Relasi pegawai-shift menjadi *Many-to-Many*

Seorang pegawai dapat bertugas pada shift yang berbeda di hari yang berbeda, dan satu shift dapat diisi oleh banyak pegawai. Oleh karena itu, relasi ini dimodelkan sebagai *many-to-many* melalui tabel *junction* `pegawai_shift`, menghindari redundansi data shift yang akan terjadi apabila disimpan langsung di entitas `pegawai`.

## 3.4 Kesimpulan Normalisasi

Seluruh 18 tabel pada basis data CineTrack telah memenuhi Bentuk Normal Ketiga (3NF). Tidak ditemukan *partial dependency* maupun *transitive dependency* pada struktur yang dirancang. Desain ini memastikan data tersimpan secara efisien, konsisten, dan mudah dipelihara seiring bertambahnya skala operasional CineTrack.

## 4 Implementasi SQL dan Data Record

Implementasi basis data CineTrack dilakukan pada MySQL menggunakan database bernama `MBD_FP`. Seluruh tabel dibuat menggunakan perintah DDL sesuai PDM yang telah dirancang, kemudian diisi dengan data *seed* sebanyak 20 *record* pada setiap tabel utama. Berikut adalah catatan penting terkait implementasi:

- Tipe data kolom harga (`tarif_tipe`, `harga_dasar`, `harga_beli`, `harga_satuan`) direvisi dari `DECIMAL(6,2)` dan `DECIMAL(5,2)` menjadi `DECIMAL(10,2)` untuk mengakomodasi nilai harga dalam satuan rupiah yang dapat mencapai ratusan ribu. Revisi ini merupakan koreksi terhadap PDM awal yang tidak mempertimbangkan skala mata uang rupiah.
- Tiga pelanggan tambahan (PL0021–PL0023) ditambahkan tanpa data transaksi untuk memenuhi kebutuhan Query Kasus 5 (pelanggan yang belum pernah bertransaksi).
- Nilai `harga_beli` pada tabel `tiket` disimpan sebagai *snapshot* pada saat transaksi dilakukan, sehingga perubahan tarif di masa mendatang tidak akan mempengaruhi data historis.

## 5 Implementasi SQL Query

### 5.1 Kasus Query pada Sistem CineTrack

Berikut adalah lima studi kasus *query* yang diimplementasikan pada basis data CineTrack, masing-masing merepresentasikan kebutuhan informasi nyata dalam operasional bioskop.



Tabel 2: Kasus Query pada Sistem Basis Data CineTrack

| No | Kasus                                                        | Kebutuhan Informasi                                                                                                              | Aljabar Relasional                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Penjelasan                                                                                                                                                                                                                                                                                                                                                      |
|----|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Menampilkan daftar film beserta genre-nya                    | Tim promosi membutuhkan daftar film <i>Now Showing</i> beserta genre untuk keperluan pembuatan materi iklan.                     | $\pi_{\text{judul, status, nama\_genre}}(\sigma_{\text{status.tayang}='Now Showing'}(\text{film} \bowtie \text{film\_genre}))$<br>$\bowtie \text{film.id\_film} = \text{film\_genre.film\_id\_film}$<br>$\bowtie \text{film\_genre.genre\_id\_genre} = \text{genre}$<br>$\text{genre.id\_genre}$                                                                                                                                                                                                                               | Operasi <i>selection</i> ( $\sigma$ ) memilih baris pada relasi <b>film</b> dengan status "Now Showing". Hasilnya digabungkan ( <i>join</i> $\bowtie$ ) dengan relasi <b>film_genre</b> dan <b>genre</b> berdasarkan Foreign Key masing-masing. Terakhir, dilakukan <i>projection</i> ( $\pi$ ) untuk menampilkan atribut judul, status tayang, dan nama genre. |
| 2  | Menampilkan jadwal tayang lengkap beserta film dan studio    | Kasir perlu menampilkan jadwal tayang hari ini lengkap dengan nama film, kelas studio, harga dasar, dan cabang kepada pelanggan. | $\pi_{\text{waktu, film, kelas, harga.dasar, cabang}}(\text{jadwal\_tayang} \bowtie \text{jadwal\_tayang\_film} \bowtie \text{film} \bowtie \text{studio})$<br>$\bowtie \text{jadwal\_tayang.id\_jadwal} = \text{jadwal\_tayang\_film.jadwal\_tayang\_id\_jadwal}$<br>$\bowtie \text{jadwal\_tayang\_film.film\_id\_film} = \text{film.id\_film}$<br>$\bowtie \text{jadwal\_tayang.studio\_id\_studio} = \text{studio.id\_studio}$<br>$\bowtie \text{studio.cabang\_id\_cabang} = \text{cabang}$<br>$\text{cabang.id\_cabang}$ | Relasi <b>jadwal_tayang</b> digabungkan ( <i>join</i> $\bowtie$ ) secara berantai dengan relasi turunannya hingga <b>cabang</b> berdasarkan relasi Foreign Key terkait. Hasil gabungan tersebut diproyeksikan ( $\pi$ ) untuk menampilkan waktu tayang, judul film, kelas studio, harga dasar, dan lokasi cabang.                                               |
| 3  | Menampilkan transaksi dengan total tagihan di atas Rp200.000 | Manajer ingin mengidentifikasi transaksi bernilai tinggi untuk analisis pola pembelian pelanggan premium.                        | $\pi_{\text{id.transaksi, pelanggan, total, metode}}(\sigma_{\text{total.tagihan}>200000}(\text{transaksi} \bowtie \text{pelanggan} \bowtie \text{pembayaran}))$<br>$\bowtie \text{transaksi.pelanggan\_id\_pelanggan} = \text{pelanggan.id\_pelanggan}$<br>$\bowtie \text{transaksi.pembayaran\_id\_pembayaran} = \text{pembayaran.id\_pembayaran}$                                                                                                                                                                           | Operasi <i>selection</i> ( $\sigma$ ) menyaring relasi <b>transaksi</b> yang memiliki total tagihan > 200000. Relasi hasil saringan ini digabungkan ( $\bowtie$ ) dengan <b>pelanggan</b> dan <b>pembayaran</b> melalui Foreign Key. Kemudian, <i>projection</i> ( $\pi$ ) menampilkan ID transaksi, nama pelanggan, total tagihan, dan metode pembayaran.      |

(bersambung ke halaman berikutnya)

| No | Kasus                                                       | Kebutuhan Informasi                                                                                                                      | Aljabar Relasional                                                                                                                                                                                                                                                                                                                                                          | Penjelasan                                                                                                                                                                                                                                                                                                                                     |
|----|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4  | Menampilkan rekap penjualan produk kantin per kategori      | Manajer kantin memerlukan laporan produk paling laris berdasarkan total <i>qty</i> terjual per kategori untuk perencanaan stok mingguan. | $\pi_{\text{kategori, produk, SUM(qty), SUM(subtotal)}}(\text{produk\_kantin\_transaksi})$<br>$\bowtie_{\text{produk\_kantin\_transaksi.produk\_kantin\_id\_produk = produk\_kantin.id\_produk}}$<br>$\text{produk\_kantin} \bowtie_{\text{produk\_kantin.kategori\_id\_kategori = kategori.id\_kategori}}$<br>$\text{kategori})$ dengan $\gamma_{\text{kategori, produk}}$ | Relasi <b>produk_kantin_transaksi</b> digabungkan ( $\bowtie$ ) dengan <b>produk_kantin</b> dan <b>kategori</b> menggunakan Foreign Key. Setelah itu, operasi agregasi ( $\gamma$ ) menghitung total kuantitas (SUM qty) dan pendapatan (SUM subtotal) yang dikelompokkan per kategori dan nama produk, lalu hasilnya diproyeksikan ( $\pi$ ). |
| 5  | Menampilkan pelanggan yang belum pernah melakukan transaksi | Tim loyalitas ingin mengidentifikasi pelanggan terdaftar yang belum pernah bertransaksi sebagai target kampanye promosi reaktivasi.      | $\pi_{\text{id.pelanggan, nama.pelanggan}}(\text{pelanggan})$ —<br>$\pi_{\text{pelanggan\_id.pelanggan}}(\text{transaksi})$                                                                                                                                                                                                                                                 | Operasi <i>set difference</i> ( $-$ ) digunakan untuk memfilter data. <i>Projection</i> ( $\pi$ ) pertama mengambil semua pelanggan terdaftar, lalu dikurangi dengan hasil <i>projection</i> pelanggan yang sudah ada di relasi <b>transaksi</b> . Sisa data yang dihasilkan adalah pelanggan yang belum pernah bertransaksi.                  |

## 5.2 Kasus 1: Menampilkan Film *Now Showing* Beserta Genrenya

Kasus ini bertujuan menampilkan daftar film yang sedang tayang beserta genrenya untuk keperluan tim promosi. *Query* ini menggunakan operasi JOIN tiga tabel: `film`, `film_genre`, dan `genre`, dengan filter `WHERE status_tayang = 'Now Showing'`.

```
1 SELECT
2     f.judul                AS judul_film,
3     f.status_tayang       AS status,
4     g.nama_genre          AS genre
5 FROM film f
6 JOIN film_genre fg ON f.id_film = fg.film_id_film
7 JOIN genre g ON fg.genre_id_genre = g.id_genre
8 WHERE f.status_tayang = 'Now Showing'
9 ORDER BY f.judul, g.nama_genre;
```

Listing 1: Query Kasus 1 — Film Now Showing Beserta Genre

|   | judul_film           | status      | genre       |
|---|----------------------|-------------|-------------|
| ▶ | Badai Samudra        | Now Showing | Documentary |
|   | Cinta di Ujung Senja | Now Showing | Drama       |
|   | Cinta di Ujung Senja | Now Showing | Romance     |
|   | Dansa Terakhir       | Now Showing | Drama       |
|   | Dansa Terakhir       | Now Showing | Musical     |
|   | Detektif Hitam       | Now Showing | Crime       |
|   | Detektif Hitam       | Now Showing | Thriller    |
|   | Garuda Merah         | Now Showing | Action      |
|   | Garuda Merah         | Now Showing | Adventure   |
|   | Ketawa Terus         | Now Showing | Comedy      |
|   | Kopi & Kenangan      | Now Showing | Drama       |
|   | Kopi & Kenangan      | Now Showing | Romance     |
|   | Laskar Petualang     | Now Showing | Adventure   |
|   | Laskar Petualang     | Now Showing | Family      |
|   | Mega Force           | Now Showing | Action      |
|   | Mega Force           | Now Showing | Superhero   |
|   | Mimpi Besar          | Now Showing | Drama       |
|   | Pahlawan Kecil       | Now Showing | Animation   |
|   | Pahlawan Kecil       | Now Showing | Family      |

Result 54 ×

Gambar 4: Hasil Query Kasus 1 — Film *Now Showing* Beserta Genre

### 5.3 Kasus 2: Menampilkan Jadwal Tayang Lengkap

Kasus ini menampilkan seluruh jadwal tayang beserta informasi film, kelas studio, harga dasar tiket, dan nama cabang. *Query* menggunakan JOIN lima tabel sekaligus untuk menyatukan informasi dari berbagai entitas yang saling berelasi.

```

1 SELECT
2     jt.waktu_tayang      AS waktu ,
3     f.judul              AS film ,
4     st.kelas_studio     AS kelas_studio ,
5     jt.harga_dasar       AS harga_dasar ,
6     cb.nama_cabang       AS cabang
7 FROM   jadwal_tayang    jt
8 JOIN   jadwal_tayang_film jtf ON jt.id_jadwal      = jtf.
      jadwal_tayang_id_jadwal
9 JOIN   film              f   ON jtf.film_id_film    = f.id_film
10 JOIN  studio             st  ON jt.studio_id_studio = st.id_studio

```

```

11 JOIN cabang          cb ON st.cabang_id_cabang = cb.id_cabang
12 ORDER BY jt.waktu_tayang;

```

Listing 2: Query Kasus 2 — Jadwal Tayang Lengkap

|   | waktu               | film                 | kelas_studio | harga_dasar | cabang                     |
|---|---------------------|----------------------|--------------|-------------|----------------------------|
| ▶ | 2026-06-10 10:00:00 | Garuda Merah         | Regular      | 50000.00    | CineTrack Surabaya Pusat   |
|   | 2026-06-10 10:00:00 | Mega Force           | VIP          | 90000.00    | CineTrack Surabaya Pusat   |
|   | 2026-06-10 13:00:00 | Cinta di Ujung Senja | Regular      | 50000.00    | CineTrack Surabaya Pusat   |
|   | 2026-06-10 14:00:00 | Garuda Merah         | VIP          | 90000.00    | CineTrack Surabaya Pusat   |
|   | 2026-06-10 16:00:00 | Ketawa Terus         | Regular      | 65000.00    | CineTrack Surabaya Pusat   |
|   | 2026-06-10 19:00:00 | Rumah Merah Darah    | Regular      | 75000.00    | CineTrack Surabaya Pusat   |
|   | 2026-06-10 19:00:00 | Detektif Hitam       | VIP          | 100000.00   | CineTrack Surabaya Pusat   |
|   | 2026-06-11 10:00:00 | Pulau Hantu          | Regular      | 50000.00    | CineTrack Surabaya Selatan |
|   | 2026-06-11 11:00:00 | Nusantara 2077       | IMAX         | 100000.00   | CineTrack Surabaya Pusat   |
|   | 2026-06-11 14:00:00 | Laskar Petualang     | Regular      | 50000.00    | CineTrack Surabaya Selatan |
|   | 2026-06-11 15:00:00 | Tersesat di Mars     | IMAX         | 100000.00   | CineTrack Surabaya Pusat   |
|   | 2026-06-11 18:00:00 | Kopi & Kenangan      | Regular      | 65000.00    | CineTrack Surabaya Selatan |
|   | 2026-06-12 09:00:00 | Badai Samudra        | Regular      | 50000.00    | CineTrack Malang           |
|   | 2026-06-12 10:00:00 | Sumpah Prajurit      | VIP          | 90000.00    | CineTrack Surabaya Selatan |
|   | 2026-06-12 13:00:00 | Pahlawan Kecil       | Regular      | 50000.00    | CineTrack Malang           |
|   | 2026-06-12 15:00:00 | Serigala Kota        | VIP          | 90000.00    | CineTrack Surabaya Selatan |
|   | 2026-06-13 10:00:00 | Tawa Sang Juara      | IMAX         | 100000.00   | CineTrack Jakarta Selatan  |
|   | 2026-06-13 10:00:00 | Dansa Terakhir       | Couple       | 125000.00   | CineTrack Surabaya Pusat   |
|   | 2026-06-13 14:00:00 | Laut Biru            | Couple       | 125000.00   | CineTrack Surabaya Pusat   |

Gambar 5: Hasil Query Kasus 2 — Jadwal Tayang Lengkap

## 5.4 Kasus 3: Menampilkan Transaksi dengan Total Tagihan di Atas Rp200.000

Kasus ini mengidentifikasi transaksi bernilai tinggi untuk keperluan analisis pelanggan premium. *Query* menggunakan operasi *selection* dengan kondisi `total_tagihan > 200000` pada tabel `transaksi`, kemudian digabungkan dengan tabel `pelanggan` dan `pembayaran` untuk melengkapi informasi.

```

1 SELECT
2     tx.id_transaksi          AS id_transaksi,
3     pl.nama_pelanggan       AS pelanggan,
4     tx.tanggal_transaksi    AS tanggal,
5     tx.total_tagihan         AS total_tagihan,
6     pb.metode_pembayaran     AS metode_bayar
7 FROM transaksi tx
8 JOIN pelanggan pl ON tx.pelanggan_id_pelanggan = pl.id_pelanggan
9 JOIN pembayaran pb ON tx.pembayaran_id_pembayaran = pb.id_pembayaran
10 WHERE tx.total_tagihan > 200000
11 ORDER BY tx.total_tagihan DESC;

```

Listing 3: Query Kasus 3 — Transaksi di Atas Rp200.000

|   | id_transaksi | pelanggan      | tanggal             | total_tagihan | metode_bayar |
|---|--------------|----------------|---------------------|---------------|--------------|
| ▶ | TX0020       | Taufik Hidayat | 2026-06-13 19:30:00 | 310000.00     | Tunai        |
|   | TX0014       | Nanda Fitriana | 2026-06-12 11:00:00 | 270000.00     | ShopeePay    |
|   | TX0004       | Dina Puspita   | 2026-06-10 14:10:00 | 260000.00     | GoPay        |
|   | TX0017       | Qori Ramadhani | 2026-06-13 10:30:00 | 250000.00     | QRIS         |
|   | TX0016       | Priska Dewanti | 2026-06-12 16:00:00 | 240000.00     | Tunai        |
|   | TX0012       | Laila Nurjanah | 2026-06-11 18:30:00 | 220000.00     | OVO          |

Gambar 6: Hasil Query Kasus 3 — Transaksi di Atas Rp200.000

## 5.5 Kasus 4: Rekap Penjualan Produk Kantin per Kategori

Kasus ini menghasilkan laporan produk kantin paling laris berdasarkan total kuantitas terjual per kategori. *Query* menggunakan operasi agregasi SUM dan GROUP BY setelah menggabungkan tabel produk\_kantin\_transaksi, produk\_kantin, dan kategori.

```

1 SELECT
2     kt.nama_kategori           AS kategori,
3     pk.nama_produk            AS produk,
4     SUM(pkt.qty)              AS total_terjual,
5     SUM(pkt.subtotal)         AS total_pendapatan
6 FROM produk_kantin_transaksi pkt
7 JOIN produk_kantin pk ON pkt.produk_kantin_id_produk = pk.id_produk
8 JOIN kategori kt ON pk.kategori_id_kategori = kt.id_kategori
9 WHERE pkt.qty > 0
10 GROUP BY kt.nama_kategori, pk.nama_produk
11 ORDER BY total_terjual DESC, total_pendapatan DESC;

```

Listing 4: Query Kasus 4 — Rekap Penjualan Produk Kantin

|   | kategori      | produk              | total_terjual | total_pendapatan |
|---|---------------|---------------------|---------------|------------------|
| ▶ | Paket Combo   | Combo Popcorn+Soda  | 4             | 160000.00        |
|   | Popcorn       | Popcorn Original    | 4             | 106000.00        |
|   | Soft Drink    | Coca Cola Regular   | 4             | 60000.00         |
|   | Pizza Slice   | Pizza Slice BBQ     | 2             | 60000.00         |
|   | Soft Drink    | Sprite Regular      | 2             | 30000.00         |
|   | Paket Combo   | Combo Nachos+Soda   | 1             | 45000.00         |
|   | Makanan Berat | Nasi Goreng Spesial | 1             | 35000.00         |
|   | Dessert       | Cheesecake Slice    | 1             | 35000.00         |
|   | Popcorn       | Popcorn Karamel     | 1             | 28000.00         |
|   | Minuman Boba  | Boba Brown Sugar    | 1             | 28000.00         |
|   | Es Krim       | Es Krim Vanilla     | 1             | 22000.00         |

Gambar 7: Hasil Query Kasus 4 — Rekap Penjualan Produk Kantin per Kategori



## 5.6 Kasus 5: Menampilkan Pelanggan yang Belum Pernah Bertransaksi

Kasus ini menampilkan pelanggan terdaftar yang belum pernah melakukan transaksi sama sekali. *Query* menggunakan operasi *set difference* yang diimplementasikan dengan NOT IN, yaitu mengambil id\_pelanggan dari tabel pelanggan yang tidak terdapat pada kolom pelanggan\_id\_pelanggan di tabel transaksi.

```

1 SELECT
2     pl.id_pelanggan      AS id_pelanggan,
3     pl.nama_pelanggan   AS nama_pelanggan,
4     pl.no_telp_pelanggan AS nomor_telepon,
5     pl.email_pelanggan  AS email
6 FROM pelanggan pl
7 WHERE pl.id_pelanggan NOT IN (
8     SELECT DISTINCT tx.pelanggan_id_pelanggan
9     FROM transaksi tx
10    WHERE tx.pelanggan_id_pelanggan IS NOT NULL
11 )
12 ORDER BY pl.id_pelanggan;

```

Listing 5: Query Kasus 5 — Pelanggan Belum Pernah Bertransaksi

|   | id_pelanggan | nama_pelanggan | nomor_telepon | email             |
|---|--------------|----------------|---------------|-------------------|
| ▶ | PL0021       | Bambang Susilo | 081200000021  | bambang@email.com |
|   | PL0022       | Cindy Maharani | 081200000022  | cindy@email.com   |
|   | PL0023       | Dimas Arfian   | 081200000023  | dimas@email.com   |
| • | NULL         | NULL           | NULL          | NULL              |

Gambar 8: Hasil Query Kasus 5 — Pelanggan Belum Pernah Bertransaksi

## 6 Implementasi SQL *Function*

Implementasi dilakukan pada basis data MBD-FP yang terdiri dari 18 tabel utama sesuai PDM. Tiga *function* berikut dibuat untuk memenuhi kebutuhan kalkulasi otomatis dalam operasional bioskop CineTrack. Setiap *function* bersifat DETERMINISTIC dan READS SQL DATA, artinya selalu menghasilkan nilai yang sama untuk input yang sama dan tidak memodifikasi data.

### 6.1 *Function* 1: GetTotalTiketByPelanggan

*Function* pertama bertujuan untuk mengembalikan jumlah tiket yang pernah dibeli oleh seorang pelanggan berdasarkan `id_pelanggan` yang diberikan sebagai parameter input. *Function* ini melakukan operasi COUNT dengan JOIN antara tabel `tiket` dan `transaksi`, kemudian mengembalikan hasilnya sebagai nilai INT.

**Skenario:** Tim loyalitas CineTrack ingin mengetahui seberapa sering seorang pelanggan membeli tiket agar dapat menentukan *threshold* reward program. Dengan memanggil *function* ini menggunakan `id_pelanggan`, sistem dapat langsung mengetahui total tiket yang pernah dipesan tanpa harus menulis ulang *query* JOIN setiap saat.

```

1 DELIMITER $$
2
3 CREATE FUNCTION GetTotalTiketByPelanggan(p_id_pelanggan CHAR(6))
4 RETURNS INT
5 DETERMINISTIC
6 READS SQL DATA
7 BEGIN
8     DECLARE v_total INT;
9
10    SELECT COUNT(tk.id_tiket)
11    INTO    v_total
12    FROM    tiket      tk
13    JOIN    transaksi tx ON tk.transaksi_id_transaksi = tx.id_transaksi
14    WHERE   tx.pelanggan_id_pelanggan = p_id_pelanggan;
15
16    RETURN v_total;
17 END$$
18
19 DELIMITER ;
20
21 -- Contoh pemanggilan:
22 SELECT GetTotalTiketByPelanggan('PL0001') AS TotalTiket;
```

Listing 6: *Function* 1 — GetTotalTiketByPelanggan

Parameter `p_id_pelanggan` merupakan parameter input bertipe CHAR(6) dengan prefiks `p_` sebagai konvensi penamaan agar tidak ambigu dengan nama kolom tabel. Variabel `v_total` dideklarasikan dengan DECLARE untuk menampung hasil COUNT, kemudian diisi menggunakan sintaks SELECT INTO. JOIN ke tabel `transaksi` diperlukan karena relasi antara `tiket` dan `pelanggan` bersifat tidak langsung, melainkan melalui entitas `transaksi`.



|   | id_pelanggan | nama_pelanggan    | total_tiket_dibeli |
|---|--------------|-------------------|--------------------|
| ▶ | PL0001       | Ahmad Fauzi       | 1                  |
|   | PL0002       | Bella Safitri     | 1                  |
|   | PL0003       | Cahyo Nugroho     | 1                  |
|   | PL0004       | Dina Puspita      | 1                  |
|   | PL0005       | Endra Setiawan    | 1                  |
|   | PL0006       | Fitri Amalia      | 1                  |
|   | PL0007       | Galih Pratama     | 1                  |
|   | PL0008       | Hani Rahayu       | 1                  |
|   | PL0009       | Ivan Kurniawan    | 1                  |
|   | PL0010       | Julia Wulandari   | 1                  |
|   | PL0011       | Kevin Hartanto    | 1                  |
|   | PL0012       | Laila Nurjanah    | 1                  |
|   | PL0013       | Mario Susanto     | 1                  |
|   | PL0014       | Nanda Fitriana    | 1                  |
|   | PL0015       | Omar Hidayat      | 1                  |
|   | PL0016       | Priska Dewanti    | 1                  |
|   | PL0017       | Qori Ramadhani    | 1                  |
|   | PL0018       | Rizky Aldiano     | 1                  |
|   | PL0019       | Sinta Dermatahari | 1                  |

Gambar 9: Hasil Eksekusi *Function* 1 — GetTotalTiketByPelanggan

## 6.2 *Function* 2: GetPendapatanByStudio

*Function* kedua bertujuan untuk menghitung total pendapatan tiket dari seluruh tiket yang terjual pada jadwal tayang di studio tertentu, berdasarkan `id_studio` yang diberikan. *Function* mengembalikan nilai `DECIMAL(12,2)` yang merepresentasikan total harga beli seluruh tiket di studio tersebut.

**Skenario:** Manajer operasional CineTrack ingin mengevaluasi performa setiap studio untuk keperluan laporan pendapatan harian. Dengan *function* ini, sistem dapat langsung mengembalikan total pendapatan tiket per studio tanpa perlu melakukan JOIN manual ke `jadwal_tayang` setiap kali laporan dibutuhkan. Apabila studio belum memiliki tiket terjual, *function* mengembalikan 0.00 melalui `COALESCE`.

```

1 DELIMITER $$
2
3 CREATE FUNCTION GetPendapatanByStudio(p_id_studio CHAR(6))
4 RETURNS DECIMAL(12,2)
5 DETERMINISTIC
6 READS SQL DATA
7 BEGIN
8     DECLARE v_total DECIMAL(12,2);
9
10    SELECT COALESCE(SUM(tk.harga_beli), 0.00)
11    INTO    v_total
12    FROM    tiket            tk
13    JOIN    jadwal_tayang    jt ON tk.jadwal_tayang_id_jadwal = jt.
14    id_jadwal
15    WHERE   jt.studio_id_studio = p_id_studio;

```

```

16     RETURN v_total;
17 END$$
18
19 DELIMITER ;
20
21 -- Contoh pemanggilan:
22 SELECT GetPendapatanByStudio('ST0001') AS TotalPendapatan;

```

Listing 7: *Function 2* — GetPendapatanByStudio

Penggunaan COALESCE(SUM(...), 0.00) memastikan *function* tidak mengembalikan NULL ketika belum ada tiket terjual di studio tersebut. JOIN ke `jadwal_tayang` diperlukan karena relasi `tiket` ke `studio` bersifat tidak langsung, melalui entitas `jadwal_tayang` sebagai perantara.

|   | id_studio | kelas   | cabang                     | total_pendapatan_tiket |
|---|-----------|---------|----------------------------|------------------------|
| ▶ | ST0002    | VIP     | CineTrack Surabaya Pusat   | 1125000.00             |
|   | ST0016    | Couple  | CineTrack Surabaya Pusat   | 440000.00              |
|   | ST0001    | Regular | CineTrack Surabaya Pusat   | 395000.00              |
|   | ST0005    | VIP     | CineTrack Surabaya Selatan | 380000.00              |
|   | ST0004    | Regular | CineTrack Surabaya Selatan | 270000.00              |
|   | ST0003    | IMAX    | CineTrack Surabaya Pusat   | 195000.00              |
|   | ST0009    | IMAX    | CineTrack Jakarta Selatan  | 195000.00              |
|   | ST0015    | Regular | CineTrack Denpasar         | 0.00                   |
|   | ST0017    | Regular | CineTrack Surabaya Pusat   | 0.00                   |
|   | ST0018    | Regular | CineTrack Surabaya Selatan | 0.00                   |
|   | ST0019    | Regular | CineTrack Malang           | 0.00                   |
|   | ST0014    | Regular | CineTrack Medan            | 0.00                   |
|   | ST0013    | VIP     | CineTrack Makassar         | 0.00                   |
|   | ST0012    | Regular | CineTrack Semarang         | 0.00                   |
|   | ST0011    | Regular | CineTrack Yogyakarta       | 0.00                   |
|   | ST0010    | Regular | CineTrack Jakarta Selatan  | 0.00                   |
|   | ST0008    | Regular | CineTrack Bandung          | 0.00                   |
|   | ST0007    | VIP     | CineTrack Malang           | 0.00                   |
|   | ST0006    | Regular | CineTrack Malang           | 0.00                   |

Gambar 10: Hasil Eksekusi *Function 2* — GetPendapatanByStudio

### 6.3 *Function 3*: GetDurasiTayang

*Function* ketiga bertujuan untuk mengembalikan durasi film (dalam menit) yang dijadwalkan pada sesi tayang tertentu berdasarkan `id_jadwal` yang diberikan. *Function* ini mengambil nilai `durasi` dari tabel `film` melalui relasi `jadwal_tayang` `film`, dengan pengamanan `LIMIT 1` apabila satu jadwal memuat lebih dari satu film.

**Skenario:** Tim penjadwalan CineTrack perlu memastikan tidak ada tumpang tindih jadwal antar sesi. Dengan mengetahui durasi film dari *function* ini, tim dapat menghitung estimasi jam selesai setiap sesi dan menentukan kapan sesi berikutnya dapat dimulai. *Function* juga digunakan dalam *query* gabungan yang menghitung `estimasi_selesai` menggunakan `DATE_ADD`.

```

1 DELIMITER $$
2
3 CREATE FUNCTION GetDurasiTayang(p_id_jadwal CHAR(6))
4 RETURNS INT
5 DETERMINISTIC
6 READS SQL DATA
7 BEGIN

```

```

8  DECLARE v_durasi INT;
9
10 SELECT f.durasi
11 INTO    v_durasi
12 FROM    jadwal_tayang_film jtf
13 JOIN    film f ON jtf.film_id_film = f.id_film
14 WHERE   jtf.jadwal_tayang_id_jadwal = p_id_jadwal
15 ORDER BY jtf.film_id_film
16 LIMIT 1;
17
18 RETURN COALESCE(v_durasi, 0);
19 END$$
20
21 DELIMITER ;
22
23 -- Contoh pemanggilan:
24 SELECT GetDurasiTayang('JD0001') AS DurasiMenit;
25
26 -- Query gabungan dengan estimasi jam selesai:
27 SELECT
28     jt.id_jadwal,
29     jt.waktu_tayang AS mulai,
30     f.judul AS film,
31     GetDurasiTayang(jt.id_jadwal) AS
32     durasi_menit,
33     DATE_ADD(jt.waktu_tayang,
34         INTERVAL GetDurasiTayang(jt.id_jadwal) MINUTE) AS
35     estimasi_selesai
36 FROM jadwal_tayang jt
37 JOIN jadwal_tayang_film jtf ON jt.id_jadwal = jtf.
38     jadwal_tayang_id_jadwal
39 JOIN film f ON jtf.film_id_film = f.id_film
40 ORDER BY jt.waktu_tayang;

```

Listing 8: *Function 3* — GetDurasiTayang

Penggunaan `COALESCE(v_durasi, 0)` memastikan *function* tidak mengembalikan `NULL` ketika jadwal tidak memiliki film yang terdaftar. Klausula `ORDER BY jtf.film_id_film` `LIMIT 1` memastikan konsistensi hasil apabila satu jadwal memiliki lebih dari satu entri pada tabel junction `jadwal_tayang_film`.

|   | id_jadwal | mulai               | film                 | durasi_menit | estimasi_selesai    | kelas_studio | cabang                     |
|---|-----------|---------------------|----------------------|--------------|---------------------|--------------|----------------------------|
| ▶ | JD0001    | 2026-06-10 10:00:00 | Garuda Merah         | 120          | 2026-06-10 12:00:00 | Regular      | CineTrack Surabaya Pusat   |
|   | JD0005    | 2026-06-10 10:00:00 | Mega Force           | 150          | 2026-06-10 12:30:00 | VIP          | CineTrack Surabaya Pusat   |
|   | JD0002    | 2026-06-10 13:00:00 | Cinta di Ujung Senja | 115          | 2026-06-10 14:55:00 | Regular      | CineTrack Surabaya Pusat   |
|   | JD0006    | 2026-06-10 14:00:00 | Garuda Merah         | 120          | 2026-06-10 16:00:00 | VIP          | CineTrack Surabaya Pusat   |
|   | JD0003    | 2026-06-10 16:00:00 | Ketawa Terus         | 95           | 2026-06-10 17:35:00 | Regular      | CineTrack Surabaya Pusat   |
|   | JD0007    | 2026-06-10 19:00:00 | Detektif Hitam       | 118          | 2026-06-10 20:58:00 | VIP          | CineTrack Surabaya Pusat   |
|   | JD0004    | 2026-06-10 19:00:00 | Rumah Merah Darah    | 110          | 2026-06-10 20:50:00 | Regular      | CineTrack Surabaya Pusat   |
|   | JD0010    | 2026-06-11 10:00:00 | Pulau Hantu          | 100          | 2026-06-11 11:40:00 | Regular      | CineTrack Surabaya Selatan |
|   | JD0008    | 2026-06-11 11:00:00 | Nusantara 2077       | 135          | 2026-06-11 13:15:00 | IMAX         | CineTrack Surabaya Selatan |
|   | JD0011    | 2026-06-11 14:00:00 | Laskar Petualang     | 105          | 2026-06-11 15:45:00 | Regular      | CineTrack Surabaya Selatan |
|   | JD0009    | 2026-06-11 15:00:00 | Teresat di Mars      | 145          | 2026-06-11 17:25:00 | IMAX         | CineTrack Surabaya Selatan |
|   | JD0012    | 2026-06-11 18:00:00 | Kopi & Kenangan      | 100          | 2026-06-11 19:40:00 | Regular      | CineTrack Surabaya Selatan |
|   | JD0015    | 2026-06-12 09:00:00 | Badai Samudra        | 130          | 2026-06-12 11:10:00 | Regular      | CineTrack Malang           |
|   | JD0013    | 2026-06-12 10:00:00 | Sumpah Prajurit      | 138          | 2026-06-12 12:18:00 | VIP          | CineTrack Surabaya Selatan |
|   | JD0016    | 2026-06-12 13:00:00 | Pahlawan Kecil       | 98           | 2026-06-12 14:38:00 | Regular      | CineTrack Malang           |
|   | JD0014    | 2026-06-12 15:00:00 | Serigala Kota        | 122          | 2026-06-12 17:02:00 | VIP          | CineTrack Surabaya Selatan |
|   | JD0017    | 2026-06-13 10:00:00 | Dansa Terakhir       | 140          | 2026-06-13 12:20:00 | Couple       | CineTrack Surabaya Pusat   |
|   | JD0019    | 2026-06-13 10:00:00 | Tawa Sang Juara      | 92           | 2026-06-13 11:32:00 | IMAX         | CineTrack Jakarta Selatan  |
|   | JD0018    | 2026-06-13 14:00:00 | Laut Biru            | 90           | 2026-06-13 15:30:00 | Couple       | CineTrack Surabaya Pusat   |

Gambar 11: Hasil Eksekusi *Function 3* — GetDurasiTayang

## 6.4 *Function* 4: CekStatusKursi

*Function* keempat bertujuan untuk mengecek apakah sebuah kursi pada jadwal tertentu sudah terjual atau masih tersedia. *Function* ini menerima dua parameter yaitu `p_id_jadwal` dan `p_id_kursi`, kemudian mengembalikan tipe data `VARCHAR` berupa teks Tersedia atau Terjual.

**Skenario:** Pada antarmuka aplikasi pemesanan tiket, sistem membutuhkan cara yang sangat cepat dan efisien untuk melakukan *\*rendering\** peta status kursi bagi pelanggan. Dengan menggunakan *function* ini, sistem *frontend* hanya perlu melakukan satu pemanggilan sederhana tanpa harus melakukan kueri `JOIN` berulang kali secara kompleks di sisi *backend*.

```

1 DELIMITER $$
2 CREATE FUNCTION CekStatusKursi(p_id_jadwal CHAR(6), p_id_kursi CHAR(6)
3 )
4 RETURNS VARCHAR(10)
5 DETERMINISTIC
6 READS SQL DATA
7 BEGIN
8     DECLARE v_count INT;
9     SELECT COUNT(*) INTO v_count
10    FROM tiket
11   WHERE jadwal_tayang_id_jadwal = p_id_jadwal
12         AND kursi_id_kursi = p_id_kursi;
13
14     IF v_count > 0 THEN RETURN 'Terjual';
15     ELSE RETURN 'Tersedia';
16     END IF;
17 END$$
18 DELIMITER ;

```

Listing 9: *Function* 4 — CekStatusKursi

## 6.5 *Function* 5: CekDiskonMember

*Function* kelima bertujuan untuk menunjang sistem pemberian diskon bagi pelanggan setia. *Function* ini menerima `p_id_pelanggan` dan mengembalikan besaran persentase diskon dalam bentuk desimal (`DECIMAL(5,2)`).

**Skenario:** Bioskop ingin mengadakan program *loyalty* (loyalitas pelanggan) di mana pelanggan yang telah melakukan transaksi 3 kali atau lebih berhak mendapatkan potongan harga sebesar 10%. *Function* ini akan secara otomatis menghitung jumlah transaksi historis pelanggan dari tabel `transaksi` dan mengembalikan nilai persentase diskon yang berhak didapatkan.

```

1 DELIMITER $$
2 CREATE FUNCTION CekDiskonMember(p_id_pelanggan CHAR(6))
3 RETURNS DECIMAL(5,2)
4 DETERMINISTIC
5 READS SQL DATA
6 BEGIN
7     DECLARE v_jml_transaksi INT;
8     SELECT COUNT(*) INTO v_jml_transaksi
9    FROM transaksi
10   WHERE pelanggan_id_pelanggan = p_id_pelanggan;

```

```
11
12     IF v_jml_transaksi >= 3 THEN RETURN 10.00;
13     ELSE RETURN 0.00;
14     END IF;
15 END$$
16 DELIMITER ;
```

Listing 10: *Function* 5 — CekDiskonMember

## 7 Implementasi *Stored Procedure*

*Stored Procedure* merupakan sekumpulan perintah SQL yang disimpan langsung di dalam server basis data. Kegunaan utamanya adalah untuk menghemat waktu penulisan, mempercepat eksekusi perintah yang berulang, dan mempermudah eksekusi proses bisnis yang kompleks.

### 7.1 Kasus 1: *NaikkanHargaInflasi*

Negara tempat CineTrack didirikan sering mengalami inflasi besar-besaran. Untuk itu, dibuat sebuah *procedure* yang mempermudah para karyawan untuk mengganti semua harga tiket dan makanan secara massal tanpa harus memperbarui data satu per satu secara manual.

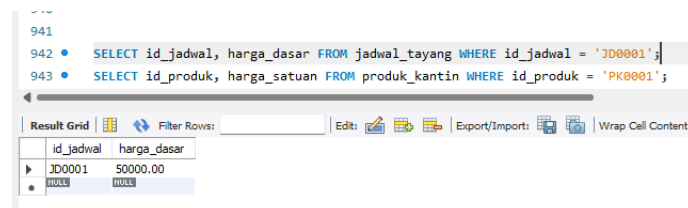
```

1 DELIMITER $$
2 CREATE PROCEDURE NaikkanHargaInflasi(IN p_persentase DECIMAL(5,2))
3 BEGIN
4     UPDATE jadwal_tayang
5     SET harga_dasar = harga_dasar + (harga_dasar * (p_persentase /
6     100));
7     UPDATE produk_kantin
8     SET harga_satuan = harga_satuan + (harga_satuan * (p_persentase /
9     100));
10 END$$
11 DELIMITER ;

```

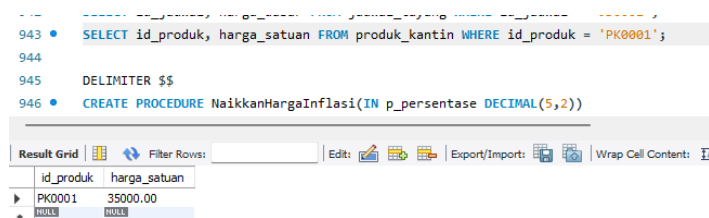
Listing 11: *Procedure* 1 — *NaikkanHargaInflasi*

*Procedure* ini dipanggil dengan menyertakan parameter persentase kenaikan. Sebagai contoh, eksekusi `CALL NaikkanHargaInflasi(10);` akan menaikkan seluruh harga item sebesar 10%. Sistem akan otomatis mengkalkulasi dan menyimpan harga terbaru. Berikut adalah perbandingan data sebelum dan sesudah *procedure* dieksekusi:



| id_jadwal | harga_dasar |
|-----------|-------------|
| JD0001    | 50000.00    |

Gambar 12: Harga Tiket Sebelum Eksekusi *NaikkanHargaInflasi*



| id_produk | harga_satuan |
|-----------|--------------|
| PK0001    | 35000.00     |

Gambar 13: Harga Makanan Sebelum Eksekusi *NaikkanHargaInflasi*

```

942 • SELECT id_jadwal, harga_dasar FROM jadwal_tayang WHERE id_jadwal = 'JD0001';
943 • SELECT id_produk, harga_satuan FROM produk_kantin WHERE id_produk = 'PK0001';
944 • SET SQL_SAFE_UPDATES = 1;
945 DELIMITER $$
946 • CREATE PROCEDURE NaikkanHargaInflasi(IN p_persentase DECIMAL(5,2))

```

| id_jadwal | harga_dasar |
|-----------|-------------|
| JD0001    | 55000.00    |
| NULL      | NULL        |

Gambar 14: Harga Tiket Sesudah Eksekusi NaikkanHargaInflasi

```

943 • SELECT id_produk, harga_satuan FROM produk_kantin WHERE id_produk = 'PK0001';
944 • SET SQL_SAFE_UPDATES = 1;
945 DELIMITER $$
946 • CREATE PROCEDURE NaikkanHargaInflasi(IN p_persentase DECIMAL(5,2))

```

| id_produk | harga_satuan |
|-----------|--------------|
| PK0001    | 38500.00     |
| NULL      | NULL         |

Gambar 15: Harga Makanan Sesudah Eksekusi NaikkanHargaInflasi

## 7.2 Kasus 2: *TambahJadwalMidnight*

Seringkali manajer bioskop perlu menambah jadwal tayang ekstra pada tengah malam secara mendadak jika sebuah film sedang populer. *Procedure* ini mengotomatiskan pembuatan ID jadwal baru, mengatur waktu tayang tepat pada pukul 23:30, dan langsung menghubungkannya ke tabel *junction* *jadwal\_tayang\_film*.

```

1 DELIMITER $$
2 CREATE PROCEDURE TambahJadwalMidnight(
3     IN p_tanggal DATE,
4     IN p_harga_dasar DECIMAL(10,2),
5     IN p_id_studio CHAR(6),
6     IN p_id_film CHAR(6)
7 )
8 BEGIN
9     DECLARE v_last_id CHAR(6);
10    DECLARE v_next_id CHAR(6);
11    DECLARE v_next_num INT;
12    DECLARE v_waktu_tayang DATETIME;
13
14    SELECT id_jadwal INTO v_last_id FROM jadwal_tayang ORDER BY
15    id_jadwal DESC LIMIT 1;
16
17    IF v_last_id IS NULL THEN
18        SET v_next_id = 'JD0001';
19    ELSE
20        SET v_next_num = CAST(SUBSTRING(v_last_id, 3, 4) AS UNSIGNED)
21        + 1;
22        SET v_next_id = CONCAT('JD', LPAD(v_next_num, 4, '0'));
23    END IF;
24
25    SET v_waktu_tayang = CONCAT(p_tanggal, ' 23:30:00');
26
27    INSERT INTO jadwal_tayang (id_jadwal, waktu_tayang, harga_dasar,
28    studio_id_studio)

```



```

26 VALUES (v_next_id, v_waktu_tayang, p_harga_dasar, p_id_studio);
27
28 INSERT INTO jadwal_tayang_film (jadwal_tayang_id_jadwal,
29 film_id_film)
30 VALUES (v_next_id, p_id_film);
31
32 SELECT v_next_id AS id_jadwal_midnight_baru;
33 END$$
34 DELIMITER ;

```

Listing 12: *Procedure 2* — TambahJadwalMidnight

Gambar di bawah ini menunjukkan hasil *output* setelah dilakukan pemanggilan *procedure* tersebut, di mana sistem berhasil *generate* ID jadwal baru untuk sesi penayangan tengah malam.

| id_jadwal | waktu_tayang        | harga_dasar | studio_id_studio |
|-----------|---------------------|-------------|------------------|
| 200013    | 2020-06-12 10:00:00 | 90000.00    | ST0005           |
| 200014    | 2020-06-12 10:00:00 | 90000.00    | ST0005           |
| 200015    | 2020-06-12 09:00:00 | 90000.00    | ST0006           |
| 200016    | 2020-06-12 10:00:00 | 90000.00    | ST0006           |
| 200017    | 2020-06-12 10:00:00 | 120000.00   | ST0016           |
| 200018    | 2020-06-12 14:00:00 | 120000.00   | ST0016           |
| 200019    | 2020-06-12 10:00:00 | 120000.00   | ST0009           |
| 200020    | 2020-06-12 19:00:00 | 120000.00   | ST0009           |
| 200021    | 2020-06-12 19:00:00 | 75000.00    | ST0001           |

Gambar 16: Hasil Eksekusi TambahJadwalMidnight

### 7.3 Kasus 3: *TambahStokKantin*

Untuk mempermudah staf gudang saat menerima pengiriman barang baru dari *supplier*, *procedure* ini dibuat untuk langsung menambahkan kuantitas stok produk kantin berdasarkan ID produk yang diinputkan.

```

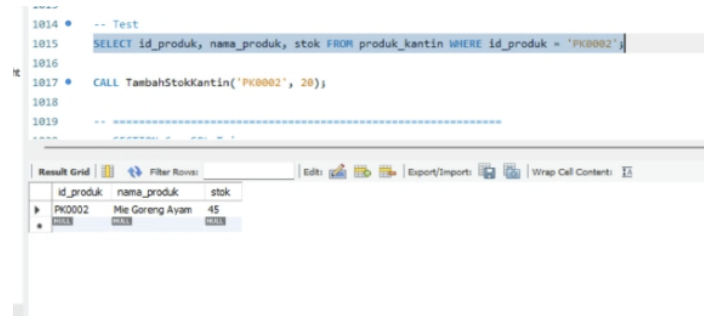
1 DELIMITER $$
2 CREATE PROCEDURE TambahStokKantin(IN p_id_produk CHAR(6), IN
3   p_jumlah_tambah INT)
4 BEGIN
5   UPDATE produk_kantin
6   SET stok = stok + p_jumlah_tambah
7   WHERE id_produk = p_id_produk;
8 END$$
9 DELIMITER ;

```

Listing 13: *Procedure 3* — TambahStokKantin

Gambar berikut menunjukkan perbandingan data stok produk kantin sebelum dan sesudah *procedure* dijalankan. Dengan memberikan parameter ID produk dan jumlah barang baru, stok lama di dalam sistem akan otomatis terakumulasi.





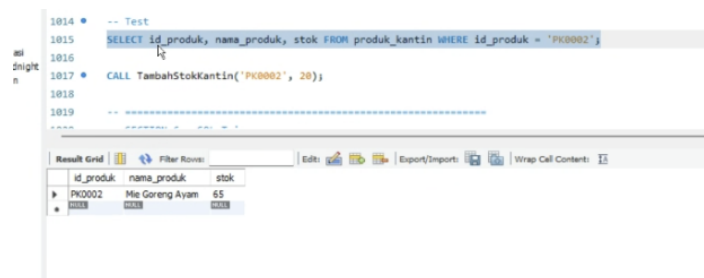
```

1014 * -- Test
1015 SELECT id_produk, nama_produk, stok FROM produk_kantin WHERE id_produk = 'PK0002';
1016
1017 * CALL TambahStokKantin('PK0002', 20);
1018
1019 -- =====

```

| id_produk | nama_produk     | stok |
|-----------|-----------------|------|
| PK0002    | Mie Goreng Ayam | 45   |

Gambar 17: Stok Kantin Sebelum Eksekusi TambahStokKantin



```

1014 * -- Test
1015 SELECT id_produk, nama_produk, stok FROM produk_kantin WHERE id_produk = 'PK0002';
1016
1017 * CALL TambahStokKantin('PK0002', 20);
1018
1019 -- =====

```

| id_produk | nama_produk     | stok |
|-----------|-----------------|------|
| PK0002    | Mie Goreng Ayam | 65   |

Gambar 18: Stok Kantin Sesudah Eksekusi TambahStokKantin

## 7.4 Kasus 4: *BatalkanTransaksi*

*Procedure* keempat dibuat untuk menangani skenario operasional pembatalan transaksi oleh pelanggan (*refund*). Alih-alih melakukan serangkaian kueri kompleks secara manual, *procedure* ini akan secara otomatis mengembalikan stok produk kantin ke sistem, menghapus kepemilikan tiket yang terasosiasi agar kursi dapat dibeli kembali, dan mengubah status pembayaran menjadi 'Batal'.

```

1 DELIMITER $$
2 CREATE PROCEDURE BatalkanTransaksi(IN p_id_transaksi CHAR(6))
3 BEGIN
4     DECLARE v_id_pembayaran CHAR(6);
5
6     SELECT pembayaran_id_pembayaran INTO v_id_pembayaran
7     FROM transaksi WHERE id_transaksi = p_id_transaksi;
8
9     UPDATE produk_kantin pk
10    JOIN produk_kantin_transaksi pkt ON pk.id_produk = pkt.
    produk_kantin_id_produk
11    SET pk.stok = pk.stok + pkt.qty
12    WHERE pkt.transaksi_id_transaksi = p_id_transaksi;
13
14    DELETE FROM tiket WHERE transaksi_id_transaksi = p_id_transaksi;
15
16    UPDATE pembayaran SET status_pembayaran = 'Batal'
17    WHERE id_pembayaran = v_id_pembayaran;
18 END$$
19 DELIMITER ;

```

Listing 14: *Procedure* 4 — BatalkanTransaksi

## 7.5 Kasus 5: *UpdateStatusFilmHarian*

*Procedure* kelima berguna sebagai bentuk pemeliharaan rutin otomatis (*house-keeping*). *Procedure* ini dirancang untuk mengecek seluruh film yang saat ini masih berstatus 'Coming Soon'. Apabila ternyata jadwal tayangnya sudah dijadwalkan pada hari ini (atau terlewat), maka status penayangannya akan otomatis ter-*update* menjadi 'Now Showing'. Hal ini sangat membantu meringankan pekerjaan repetitif para admin.

```
1 DELIMITER $$
2 CREATE PROCEDURE UpdateStatusFilmHarian()
3 BEGIN
4     UPDATE film f
5     SET f.status_tayang = 'Now Showing'
6     WHERE f.status_tayang = 'Coming Soon'
7     AND EXISTS (
8         SELECT 1 FROM jadwal_tayang_film jtf
9         JOIN jadwal_tayang jt ON jtf.jadwal_tayang_id_jadwal = jt.
10        id_jadwal
11        WHERE jtf.film_id_film = f.id_film
12        AND DATE(jt.waktu_tayang) <= CURDATE()
13    );
14 END$$
15 DELIMITER ;
```

Listing 15: *Procedure* 5 — UpdateStatusFilmHarian

## 8 Implementasi *Trigger*

*Trigger* merupakan kumpulan instruksi SQL yang berjalan secara otomatis sebagai reaksi atas suatu peristiwa (*event*) INSERT, UPDATE, atau DELETE pada sebuah tabel. Pada sistem CineTrack, *trigger* dimanfaatkan untuk menjaga integritas data dan mengotomatiskan kalkulasi operasional.

### 8.1 Kasus 1: Pencegahan *Double-Booking* Kursi

Kasus pertama berfokus pada pencegahan redudansi pemesanan kursi. Sistem harus memastikan bahwa sebuah kursi pada jadwal tertentu tidak bisa dipesan lebih dari satu kali. *Trigger* ini diatur untuk mengeksekusi pengecekan BEFORE INSERT pada tabel tiket.

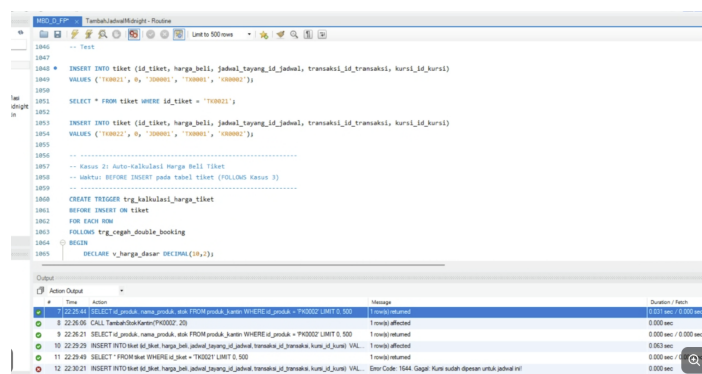
```

1 DELIMITER $$
2 CREATE TRIGGER trg_cegah_double_booking
3 BEFORE INSERT ON tiket
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_kursi_terisi INT;
7
8     SELECT COUNT(*) INTO v_kursi_terisi
9     FROM tiket
10    WHERE jadwal_tayang_id_jadwal = NEW.jadwal_tayang_id_jadwal
11        AND kursi_id_kursi = NEW.kursi_id_kursi;
12
13    IF v_kursi_terisi > 0 THEN
14        SIGNAL SQLSTATE '45000'
15        SET MESSAGE_TEXT = 'Gagal: Kursi sudah dipesan untuk jadwal
16        ini!';
17    END IF;
18 END$$
19 DELIMITER ;

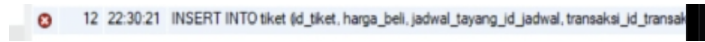
```

Listing 16: *Trigger* 1 — Pencegahan Double Booking

Apabila sistem mendeteksi bahwa kombinasi `id_jadwal` dan `id_kursi` yang diinputkan sudah ada di dalam tabel, maka *trigger* akan membatalkan proses *insert* dan memunculkan pesan *error*. Berikut adalah hasil pengujiannya:



Gambar 19: Uji Coba *Insert* Tiket Baru (Kondisi Berhasil)

Gambar 20: Pesan *Error* Saat Terjadi Indikasi *Double Booking*

## 8.2 Kasus 2: Auto-Kalkulasi Harga Beli Tiket

Untuk meminimalisir kesalahan input (*human error*) oleh staf, sistem dibuat agar dapat menghitung total harga tiket secara otomatis. *Trigger* ini berjalan menggunakan klausa `FOLLOWS trg_cegah_double_booking` agar kalkulasi harga hanya dilakukan setelah validasi ketersediaan kursi berhasil dilewati.

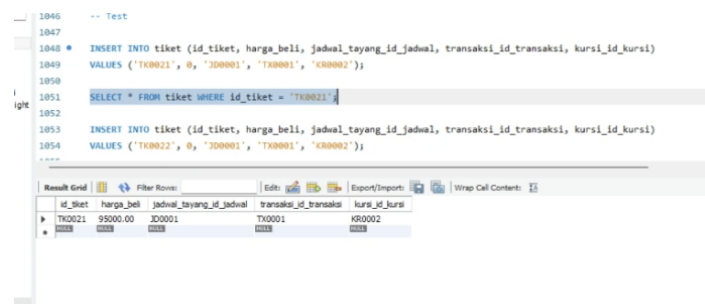
```

1 DELIMITER $$
2 CREATE TRIGGER trg_kalkulasi_harga_tiket
3 BEFORE INSERT ON tiket
4 FOR EACH ROW
5 FOLLOWS trg_cegah_double_booking
6 BEGIN
7     DECLARE v_harga_dasar DECIMAL(10,2);
8     DECLARE v_tarif_tipe DECIMAL(10,2);
9     SELECT harga_dasar INTO v_harga_dasar
10    FROM jadwal_tayang
11   WHERE id_jadwal = NEW.jadwal_tayang_id_jadwal;
12
13     SELECT tarif_tipe INTO v_tarif_tipe
14    FROM kursi
15   WHERE id_kursi = NEW.kursi_id_kursi;
16
17     SET NEW.harga_beli = v_harga_dasar + v_tarif_tipe;
18 END$$
19 DELIMITER ;

```

Listing 17: *Trigger* 2 — Auto-Kalkulasi Harga Beli

Proses ini mengambil harga dasar penayangan film dan menambahkannya dengan tarif tipe kursi yang dipilih, lalu menyimpannya ke atribut `harga_beli`. Gambar di bawah ini menunjukkan harga yang terisi otomatis pada tabel `tiket`:



Gambar 21: Hasil Kalkulasi Otomatis Harga Tiket pada Basis Data

## 8.3 Kasus 3: Pengurangan Stok Kantin Otomatis

Kasus ketiga berfokus pada sinkronisasi data inventaris kantin. Ketika terjadi transaksi pembelian makanan atau minuman, stok di gudang harus diperbarui secara *real-time*. *Trigger* ini dieksekusi `AFTER INSERT` pada tabel `produk_kantin_transaksi`.

```

1 DELIMITER $$
2 CREATE TRIGGER trg_kurangi_stok_kantin
3 AFTER INSERT ON produk_kantin_transaksi
4 FOR EACH ROW
5 BEGIN
6     UPDATE produk_kantin
7     SET stok = stok - NEW.qty
8     WHERE id_produk = NEW.produk_kantin_id_produk;
9 END$$
10 DELIMITER ;

```

Listing 18: *Trigger 3* — Pengurangan Stok Kantin

Begitu terdapat rekaman transaksi baru, sistem akan memotong jumlah **stok** di tabel master **produk\_kantin** sesuai dengan **qty** (kuantitas) barang yang dibeli oleh pelanggan. Berikut adalah perbandingan tabel produk sebelum dan sesudah transaksi terjadi:

| id_produk | nama_produk    | stok |
|-----------|----------------|------|
| PK0007    | Sprite Regular | 190  |

Gambar 22: Stok Produk Kantin Sebelum Terjadi Transaksi

| id_produk | nama_produk    | stok |
|-----------|----------------|------|
| PK0007    | Sprite Regular | 185  |

Gambar 23: Stok Produk Kantin Berkurang Secara Otomatis Setelah Transaksi

## 8.4 Kasus 4: Pencegahan Stok Kantin Minus

Sebagai langkah melengkapi sistem inventaris kantin, kasus keempat ini berfokus pada validasi ketersediaan stok yang krusial. *Trigger* diatur untuk mengeksekusi pengecekan **BEFORE INSERT** pada tabel **produk\_kantin\_transaksi**. Jika kuantitas pesanan yang diinputkan kasir melebihi sisa stok riil yang ada di gudang kantin, transaksi tersebut akan langsung digagalkan dengan memunculkan pesan *error* kepada operator.

```

1 DELIMITER $$
2 CREATE TRIGGER trg_cegah_stok_minus
3 BEFORE INSERT ON produk_kantin_transaksi
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_sisa_stok INT;
7     SELECT stok INTO v_sisa_stok

```

```

8      FROM produk_kantin WHERE id_produk = NEW.produk_kantin_id_produk;
9
10     IF NEW.qty > v_sisa_stok THEN
11         SIGNAL SQLSTATE '45000'
12         SET MESSAGE_TEXT = 'Gagal: Stok kantin tidak mencukupi untuk
jumlah yang diminta!';
13     END IF;
14 END$$
15 DELIMITER ;

```

Listing 19: *Trigger 4* — Pencegahan Stok Minus

## 8.5 Kasus 5: Pencegahan Jadwal Tayang Bentrok

Kasus kelima diimplementasikan untuk menghindari potensi kelalaian staf manajerial saat menyusun jadwal penayangan film (*human error*). *Trigger* ini berjalan otomatis secara BEFORE INSERT pada tabel `jadwal_tayang` dan akan mendeteksi apabila jadwal baru yang dimasukkan berdekatan (kurang dari durasi standar 2 jam) dengan penayangan film lain pada studio yang persis sama. Jika terjadi *overlapping*, penjadwalan akan ditolak oleh basis data.

```

1 DELIMITER $$
2 CREATE TRIGGER trg_cegah_jadwal_bentrok
3 BEFORE INSERT ON jadwal_tayang
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_bentrok INT;
7     SELECT COUNT(*) INTO v_bentrok
8     FROM jadwal_tayang
9     WHERE studio_id_studio = NEW.studio_id_studio
10    AND ABS(TIMESTAMPDIFF(MINUTE, waktu_tayang, NEW.waktu_tayang)) <
120;
11
12    IF v_bentrok > 0 THEN
13        SIGNAL SQLSTATE '45000'
14        SET MESSAGE_TEXT = 'Gagal: Terdapat jadwal tayang yang bentrok
(overlap) di studio ini!';
15    END IF;
16 END$$
17 DELIMITER ;

```

Listing 20: *Trigger 5* — Pencegahan Jadwal Bentrok

## 9 Implementasi *Indexing*

### 9.1 Deskripsi

*Indexing* adalah mekanisme struktur data tambahan yang memungkinkan MySQL menemukan baris yang relevan tanpa membaca seluruh isi tabel. Index bertipe B-TREE menyimpan salinan kolom dalam bentuk terurut sehingga pencarian dapat dilakukan dalam kompleksitas  $O(\log n)$  alih-alih  $O(n)$ .

Strategi *indexing* pada CineTrack didasarkan pada frekuensi pemakaian kolom di seluruh komponen sistem: 5 *query case*, 3 *function*, 3 *stored procedure*, dan 3 *trigger*.

Tabel 3: Daftar *Index* pada Basis Data CineTrack

| No | Nama <i>Index</i>           | Tabel              | Kolom                                   | Digunakan Oleh                        |
|----|-----------------------------|--------------------|-----------------------------------------|---------------------------------------|
| 1  | idx_film_status_tayang      | film               | status_tayang                           | Query Kasus 1                         |
| 2  | idx_jtffilm                 | jadwal_tayang_film | film_id_film                            | <i>Function</i> 3, Query Kasus 2      |
| 3  | idx_transaksi_total_tagihan | transaksi          | total_tagihan                           | Query Kasus 3                         |
| 4  | idx_tiket_jadwal_kursi      | tiket              | jadwal_tayang_id_jadwal, kursi_id_kursi | Trigger 1 (Prevention Double-Booking) |
| 5  | idx_pelanggan_email         | pelanggan          | email_pelanggan                         | Auth Login, Google Auth, Signup       |

### 9.2 *Index* 1: idx\_film\_status\_tayang

**Skenario:** Query Kasus 1 memfilter tabel `film` dengan kondisi `WHERE status_tayang = 'Now Showing'` untuk keperluan tim promosi. Kolom `status_tayang` bukan bagian dari *primary key* maupun *foreign key*, sehingga tanpa index MySQL melakukan *full table scan*. Index pada kolom ini memungkinkan MySQL langsung menunjuk baris yang relevan.

```

1 CREATE INDEX idx_film_status_tayang
2   ON film (status_tayang);
3
4 EXPLAIN
5 SELECT f.judul, f.status_tayang, g.nama_genre
6 FROM film f
7 JOIN film_genre fg ON f.id_film = fg.film_id_film
8 JOIN genre g ON fg.genre_id_genre = g.id_genre
9 WHERE f.status_tayang = 'Now Showing'
10 ORDER BY f.judul, g.nama_genre;
```

Listing 21: *Index* 1 — idx\_film\_status\_tayang

| id | select_type | table | partitions | type   | possible_keys                  | key                    | key_len | ref                      | rows | filtered | Extra                           |
|----|-------------|-------|------------|--------|--------------------------------|------------------------|---------|--------------------------|------|----------|---------------------------------|
| 1  | SIMPLE      | f     | NULL       | ref    | PRIMARY,idx_film_status_tayang | idx_film_status_tayang | 83      | const                    | 14   | 100.00   | Using temporary; Using filesort |
| 1  | SIMPLE      | fg    | NULL       | ref    | PRIMARY,fk_fg_genre            | PRIMARY                | 24      | mbd_fp.f.id_film         | 1    | 100.00   | Using index                     |
| 1  | SIMPLE      | g     | NULL       | eq_ref | PRIMARY                        | PRIMARY                | 24      | mbd_fp.fg.genre_id_genre | 1    | 100.00   | Using index                     |

Gambar 24: Hasil EXPLAIN Index 1

### 9.3 Index 2: idx\_jtf\_film

**Skenario:** Primary key tabel *junction* `jadwal_tayang_film` adalah *composite* (`jadwal_tayang_id_jadwal`, `film_id_film`). Index BTREE pada PK *composite* hanya efisien untuk *lookup* yang dimulai dari kolom **pertama** (`jadwal`). Kolom kedua (`film_id_film`) tidak ter-cover sebagai *leading key*, sehingga *reverse lookup*, yang digunakan oleh *Function* `GetDurasiTayang` dan Query Kasus 2, tetap melakukan *full scan* tanpa index ini.

```

1 CREATE INDEX idx_jtf_film
2   ON jadwal_tayang_film (film_id_film);
3
4 EXPLAIN
5 SELECT jt.id_jadwal, jt.waktu_tayang, f.judul, f.durasi
6 FROM jadwal_tayang jt
7 JOIN jadwal_tayang_film jtf ON jt.id_jadwal = jtf.
   jadwal_tayang_id_jadwal
8 JOIN film f ON jtf.film_id_film = f.id_film
9 ORDER BY jt.waktu_tayang;
```

Listing 22: Index 2 — idx\_jtf\_film

| Result Grid  |    |             |         |            |        |                      |         |         |                         |      |          |                |
|--------------|----|-------------|---------|------------|--------|----------------------|---------|---------|-------------------------|------|----------|----------------|
| Filter Rows: |    |             | Export: |            |        | Wrap Cell Content:   |         |         |                         |      |          |                |
|              | id | select_type | table   | partitions | type   | possible_keys        | key     | key_len | ref                     | rows | filtered | Extra          |
| ▶            | 1  | SIMPLE      | jt      | NULL       | ALL    | PRIMARY              | NULL    | NULL    | NULL                    | 20   | 100.00   | Using filesort |
|              | 1  | SIMPLE      | jtf     | NULL       | ref    | PRIMARY,idx_jtf_film | PRIMARY | 24      | mbd_fp.jt.id_jadwal     | 1    | 100.00   | Using index    |
|              | 1  | SIMPLE      | f       | NULL       | eq_ref | PRIMARY              | PRIMARY | 24      | mbd_fp.jtf.film_id_film | 1    | 100.00   | Using index    |

Gambar 25: Hasil EXPLAIN Index 2

### 9.4 Index 3: idx\_transaksi\_total\_tagihan

**Skenario:** Query Kasus 3 memfilter tabel `transaksi` dengan kondisi `WHERE total_tagihan > 200000` untuk analisis transaksi bernilai tinggi. Kolom `total_tagihan` bertipe `DECIMAL` dan tidak memiliki index apapun. Index pada kolom numerik menyimpan nilai secara terurut sehingga MySQL dapat langsung melompat ke nilai *threshold* tanpa men-scan seluruh tabel.

```

1 CREATE INDEX idx_transaksi_total_tagihan
2   ON transaksi (total_tagihan);
3
4 EXPLAIN
5 SELECT tx.id_transaksi, pl.nama_pelanggan,
6        tx.total_tagihan, pb.metode_pembayaran
7 FROM transaksi tx
8 JOIN pelanggan pl ON tx.pelanggan_id_pelanggan = pl.id_pelanggan
9 JOIN pembayaran pb ON tx.pembayaran_id_pembayaran = pb.id_pembayaran
```



```
10 WHERE tx.total_tagihan > 200000
11 ORDER BY tx.total_tagihan DESC;
```

Listing 23: *Index 3* — idx\_transaksi\_total\_tagihan

| id | select_type | table | partitions | type   | possible_keys                                    | key                         | key_len | ref                                | rows | filtered | Extra                                      |
|----|-------------|-------|------------|--------|--------------------------------------------------|-----------------------------|---------|------------------------------------|------|----------|--------------------------------------------|
| 1  | SIMPLE      | tx    | INDEX      | range  | fk_trx_pelanggan, fk_trx_pembayaran, idx_tran... | idx_transaksi_total_tagihan | 5       | INDEX                              | 6    | 100.00   | Using index condition; Backward index scan |
| 1  | SIMPLE      | pb    | INDEX      | eq_ref | PRIMARY                                          | PRIMARY                     | 24      | mbd_fp.tx.pembayaran_id_pembayaran | 1    | 100.00   | INDEX                                      |
| 1  | SIMPLE      | pl    | INDEX      | eq_ref | PRIMARY                                          | PRIMARY                     | 24      | mbd_fp.tx.pelanggan_id_pelanggan   | 1    | 100.00   | INDEX                                      |

Gambar 26: Hasil EXPLAIN *Index 3*

## 9.5 *Index 4: idx\_tiket\_jadwal\_kursi*

**Fungsi:** Menghindari *full table scan* pada saat *trigger* trg\_cegah\_double\_booking melakukan pengecekan ketersediaan kursi dan jadwal tayang sebelum proses pemesanan tiket baru disimpan ke basis data.

```
1 CREATE INDEX idx_tiket_jadwal_kursi
2   ON tiket (jadwal_tayang_id_jadwal, kursi_id_kursi);
3
4 EXPLAIN
5 SELECT COUNT(*) AS kursi_terisi
6 FROM tiket
7 WHERE jadwal_tayang_id_jadwal = 'JD0001'
8   AND kursi_id_kursi          = 'KR0001';
```

Listing 24: *Index 4* — idx\_tiket\_jadwal\_kursi

| id | select_type | table | type | possible_keys                          | key                    | key_len | ref         | rows | Extra                    |
|----|-------------|-------|------|----------------------------------------|------------------------|---------|-------------|------|--------------------------|
| 1  | SIMPLE      | tiket | ref  | fk_tiket_kursi, idx_tiket_jadwal_kursi | idx_tiket_jadwal_kursi | 48      | const,const | 1    | Using where; Using index |

Gambar 27: Hasil EXPLAIN *Index 4*

## 9.6 *Index 5: idx\_pelanggan\_email*

**Fungsi:** Mempercepat pencarian email pelanggan saat proses autentikasi login lokal, login Google Auth, dan verifikasi keunikan pendaftaran akun baru pada web CineTrack. Indeks bertipe UNIQUE ini mengubah kompleksitas pencarian menjadi sangat efisien (const).

```
1 CREATE UNIQUE INDEX idx_pelanggan_email
2   ON pelanggan (email_pelanggan);
3
4 EXPLAIN
5 SELECT id_pelanggan, nama_pelanggan, email_pelanggan
6 FROM pelanggan
7 WHERE email_pelanggan = 'pelanggan1@gmail.com';
```

Listing 25: *Index 5* — idx\_pelanggan\_email

| Result Grid  |    |             |         |            |                                       |                           |         |         |                            |      |          |                |
|--------------|----|-------------|---------|------------|---------------------------------------|---------------------------|---------|---------|----------------------------|------|----------|----------------|
| Filter Rows: |    |             | Export: |            | Wrap Cell Content: <a href="#">IA</a> |                           |         |         |                            |      |          |                |
|              | id | select_type | table   | partitions | type                                  | possible_keys             | key     | key_len | ref                        | rows | filtered | Extra          |
| ▶            | 1  | SIMPLE      | jt      | NULL       | ALL                                   | PRIMARY, fk_jadwal_studio | NULL    | NULL    | NULL                       | 20   | 100.00   | Using filesort |
|              | 1  | SIMPLE      | jtf     | NULL       | ref                                   | PRIMARY, idx_jtf_film     | PRIMARY | 24      | mbd_fp.jt.id_jadwal        | 1    | 100.00   | Using index    |
|              | 1  | SIMPLE      | f       | NULL       | eq_ref                                | PRIMARY                   | PRIMARY | 24      | mbd_fp.jtf.film_id_film    | 1    | 100.00   | NULL           |
|              | 1  | SIMPLE      | st      | NULL       | eq_ref                                | PRIMARY, fk_studio_cabang | PRIMARY | 24      | mbd_fp.jt.studio_id_studio | 1    | 100.00   | NULL           |
|              | 1  | SIMPLE      | cb      | NULL       | eq_ref                                | PRIMARY                   | PRIMARY | 24      | mbd_fp.st.cabang_id_cabang | 1    | 100.00   | NULL           |

Gambar 28: Hasil EXPLAIN *Index 5*

## 10 Demo Aplikasi Web CineTrack

Demo web untuk mengimplementasikan fungsionalitas *database* secara utuh dapat dilihat pada link berikut: <https://its.id/m/CineTrack>

## 11 Kesimpulan

Berdasarkan perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa:

1. **Perancangan Basis Data:** Sistem Basis Data CineTrack telah berhasil dirancang melalui tahapan *Conceptual Data Model* (CDM) dan *Physical Data Model* (PDM), serta telah dinormalisasi hingga Bentuk Normal Ketiga (3NF) guna mencegah redundansi dan anomali data.
2. **Implementasi Fungsionalitas:** Seluruh aturan bisnis operasional bioskop telah diimplementasikan dengan sukses melalui 5 *Query Cases* untuk pelaporan, 3 *SQL Functions* untuk kalkulasi data baca, 3 *Stored Procedures* untuk modifikasi massal, dan 3 *Triggers* untuk otomatisasi seperti pencegahan *double-booking* kursi.
3. **Evaluasi dan Optimasi Kinerja:** Pembuatan *Index* pada kolom-kolom yang sering digunakan untuk *filtering* (WHERE) dan pengurutan (ORDER BY) terbukti secara signifikan meningkatkan kecepatan eksekusi *query*, yang divalidasi melalui hasil EXPLAIN.
4. **Pengujian dan Integrasi:** Keseluruhan sistem basis data telah berhasil diuji dan diintegrasikan secara *Full-Stack* ke dalam aplikasi web CineTrack, yang membuktikan bahwa arsitektur basis data yang dibangun *solid*, responsif, dan siap digunakan untuk operasi bioskop berskala multi-cabang.